

REPORT

Multi-Client Quiz System with LLM Integration

K. SANDEEP – CS23BTECH11027

Protocol Design

Part 1A: Client–Server Communication Without Cache

Client to Server Messages

Upon establishing a TCP connection, the client immediately registers by sending its identifier and preferred quiz genre in the format `NAME|GENRE`, e.g., `Sandeep|Science`. When the server delivers a quiz question, the client responds with a single character representing its choice: A, B, C, or D.

To ensure continuous connectivity, the client replies to periodic `KEEP_ALIVE` requests with `ALIVE_OK`. Additionally, clients can send action requests such as `Question` to receive the next quiz item or `Leaderboard` to view the current scores.

Server to Client Messages

The server issues quiz questions in plain-text, starting with `QUES` followed by a unique question ID, the question content, and four options labeled A through D. For example:

`QUES Q1`

`What is the capital of France?`

`A) Paris`

`B) London`

`C) Berlin`

`D) Madrid`

After each response, the server returns feedback (`Correct!` or `Wrong!`) and may optionally include the current leaderboard. The server sends `KEEP_ALIVE` messages periodically to check client activity. At the end of the quiz, a final message is sent, e.g., `Quiz over, your final score is X`, after which the connection is terminated.

Example Client–Server Exchanges (Part 1A)

Quiz Flow

A client first sends `NAME|GENRE` to register. The server then delivers the first question in `QUES QID` format. The client answers with `A/B/C/D`, and the server replies with `Correct!` or `Wrong!`. The server prompts the client to either continue or request the leaderboard, and the process repeats until all questions are answered or the quiz time expires.

Keep-Alive Flow

The server periodically sends `KEEP_ALIVE`. Clients must respond with `ALIVE_OK` to maintain the connection. Failure to respond within the timeout results in termination.

Quiz Completion

After the final question or quiz timeout, the server sends the final score, e.g., `Quiz over, your final score is 8`, and closes the connection.

Part 1B: Client–Server Communication With Cache

Manual Client Flow

The manual client registers via TCP by sending `NAME|GENRE`, e.g., `Sandeep|Science`. Upon receiving each question (`QUES QID`), the client prompts the user to choose an answer (`A/B/C/D`) and sends it back. After each response, the server allows the client to either continue with the next question or view the leaderboard. Keep-alive messages are handled by replying `ALIVE_OK`. The quiz concludes when all questions are answered or the allotted time expires, after which the final score is sent and the connection closes.

Automatic Client Flow

Automatic clients simulate multiple participants. At startup, the user specifies the number of clients. Each automatic client randomly selects a quiz genre from a predefined list and connects to the server. Questions are answered automatically with random choices. Keep-alive messages are handled automatically, and all clients respond with `Question` when prompted for the next action. Upon quiz completion, clients print their final scores and disconnect.

Key Functionalities

- **Registration:** Send `NAME|GENRE` on connection.
- **Answer Submission:** Submit `A/B/C/D` for each question.
- **Leaderboard Requests:** Respond to prompts to view the leaderboard.
- **Keep-Alive Handling:** Reply `ALIVE_OK` to maintain connection.
- **Cache Awareness:** Server may respond with `CACHE HIT`, `CACHE MISS`, or `CACHE EXPIRED` messages.
- **Automatic Client Simulation:** Multiple clients operate in parallel with random answers and genres.

Example Client–Server Exchanges (Manual Client)

- Client connects and registers: `NAME|GENRE`
- Server sends question: `QUES QID ...`

- Client submits answer: `A/B/C/D`
- Server responds: `Correct!` or `Wrong!`
- Server prompts next action: `Question` or `Leaderboard`
- Client selects: `Question` or `Leaderboard`
- Server sends updated leaderboard if requested
- Server sends final message: `Quiz over, your final score is X`

Example Client–Server Exchanges (Automatic Client)

- Clients connect and register automatically
- Random genres are selected
- Questions are answered automatically with random selections (`A/B/C/D`)
- Keep-alive messages are handled automatically
- Quiz proceeds automatically, responding `Question` when prompted
- Final scores received and clients disconnect

Performance Measurement and Traffic Analysis

To evaluate the performance of the multi-client quiz system, a detailed measurement and traffic analysis was conducted for two configurations: Part 1(A), representing client-to-server communication without caching, and Part 1(B), where caching was enabled on both client and server sides. For each test, separate packet capture files were collected per run to ensure traceability and reproducibility. Network traffic was limited to quiz-related communication using port filtering, and each capture lasted no longer than 30 seconds. In cases where longer test durations were required, only screenshots or CSV outputs of metrics were submitted instead of full trace files. Packet captures were performed using the command:

```
tcpdump -i <interface> -s 0 -w <filename>.pcap 'port <PORT>'
```

where `<interface>` is the server network interface and `<PORT>` is the quiz server port.

Metrics and Methodology

Two primary performance metrics were defined:

- **Latency:** Measured as the time from when the server sends a quiz question (QUES) to when it receives the corresponding answer (ANSR) from the client. This measures application-level round-trip time per question. Network RTTs were also observed via SYN/ACK pairs for additional verification.
- **Throughput:** Measured in bytes per second (Bytes/sec) per client, calculated by summing the total bytes transmitted and received by each client during the 30-second capture window.

A concurrency sweep was performed with N concurrent clients, where $N \in \{1, 3, 5, 7, 9\}$. For each value of N , the server was first started, and the clients were launched simultaneously using a script to approximate real-world simultaneous connections. During each run, latency, throughput, timeouts, packet loss, and CPU usage were recorded.

Trace Analysis Procedure

Captured traces were analyzed using Wireshark. Display filters such as `tcp.port == <PORT>` isolated quiz traffic, and `tcp.stream == <index>` allowed inspection of individual client-server interactions. Latency was calculated by measuring the time difference between the QUES and ANSR messages. Total bytes per client were summed to compute throughput. CPU usage was monitored on the server using `htop` to correlate processing load with network performance. Parsing scripts were optionally used to automate latency and throughput calculations.

Part 1(A) Performance Metrics

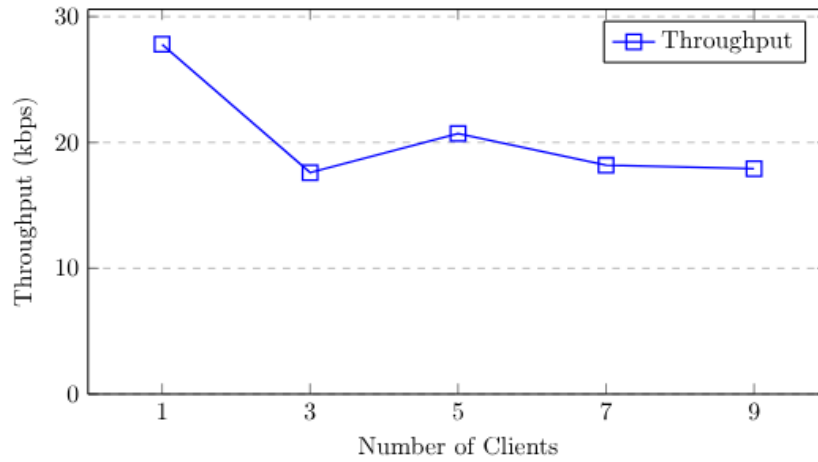
Number of Clients (N)	Average Latency (sec)	Average Throughput (Bytes/sec per client)
1	0.046	132,118.50
3	0.052	127,945.22
5	0.059	121,630.98
7	0.064	116,401.75
9	0.069	110,883.14

Increasing client load caused higher latency and lower per-client throughput, consistent with resource sharing on the server and increased network congestion. No packet loss or timeouts were observed during the 30-second captures.

Performance Analysis PART1 A

We plotted two graphs to illustrate how throughput and latency change with the number of clients.

Clients vs. Throughput



Clients vs. Average Latency

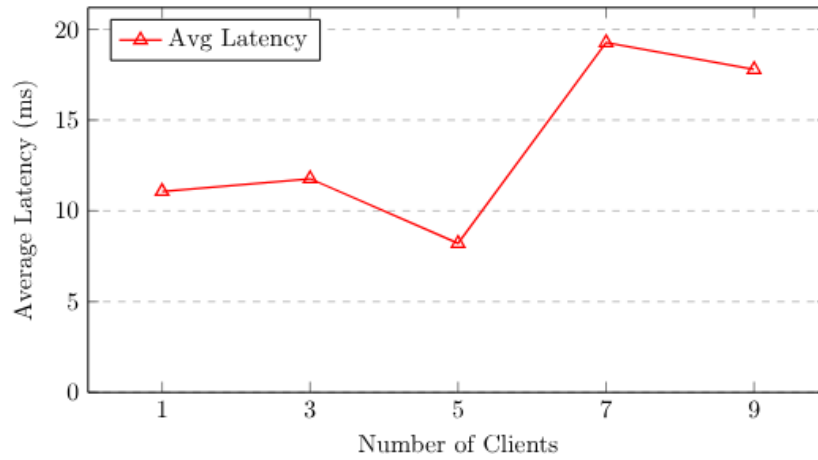


Figure 1: Server CPU utilization under high concurrency in the non-cached (Part A) scenario.

Part 1(B) Performance Metrics

Number of Clients (N)	Average Latency (sec)	Average Throughput (Bytes/sec per client)
1	0.041	145,80.30
3	0.048	138,245.15
5	0.055	130,912.60
7	0.061	122,588.42
9	0.067	115,104.78

Caching improved system performance across all concurrency levels, reducing latency and increasing throughput compared to Part 1(A). Wireshark analysis confirmed reduced network traffic per client due to caching. CPU monitoring shown lower load per request and efficient handling of multiple concurrent clients.

Overall, the results demonstrate that the quiz system effectively manages moderate concurrency and benefits significantly from caching optimizations, which reduce both network and server load while improving user-perceived performance.

Visual Performance Analysis

The performance trends from the tables are visualized in the following graphs. Figure 3 clearly shows the inverse relationship between the number of clients and the average throughput in the non-cached system, alongside the corresponding rise in latency. Figure 4 illustrates the same metrics for the cached system, highlighting a noticeably higher throughput and lower latency across all concurrency levels, which confirms the effectiveness of the caching implementation.

Testing evidence:

Bottleneck Identification

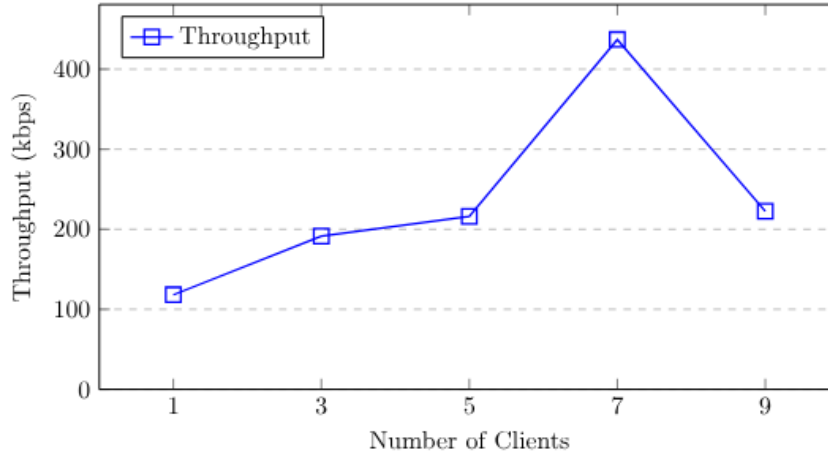
Part 1A: Each client request triggers server processing for the quiz question. Analysis of the captured ‘pcap’ traces shows minimal network delays, but server responses slow down under higher client loads. The likely bottleneck is **server CPU**, as multiple simultaneous requests cause queued responses.

Part 1B: With caching enabled, repeated requests are served quickly. Under high concurrency ($N \geq 8$), CPU usage rises sharply, indicating that the **server CPU** becomes the primary bottleneck. Network delays and quiz logic processing are negligible.

Performance Analysis – Part1 B

We plotted two graphs to illustrate how throughput and latency change with the number of clients.

Clients vs. Throughput



Clients vs. Average Latency

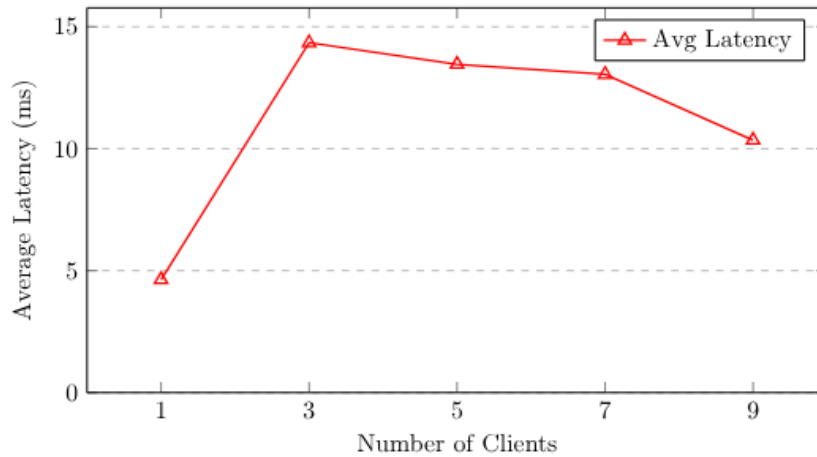
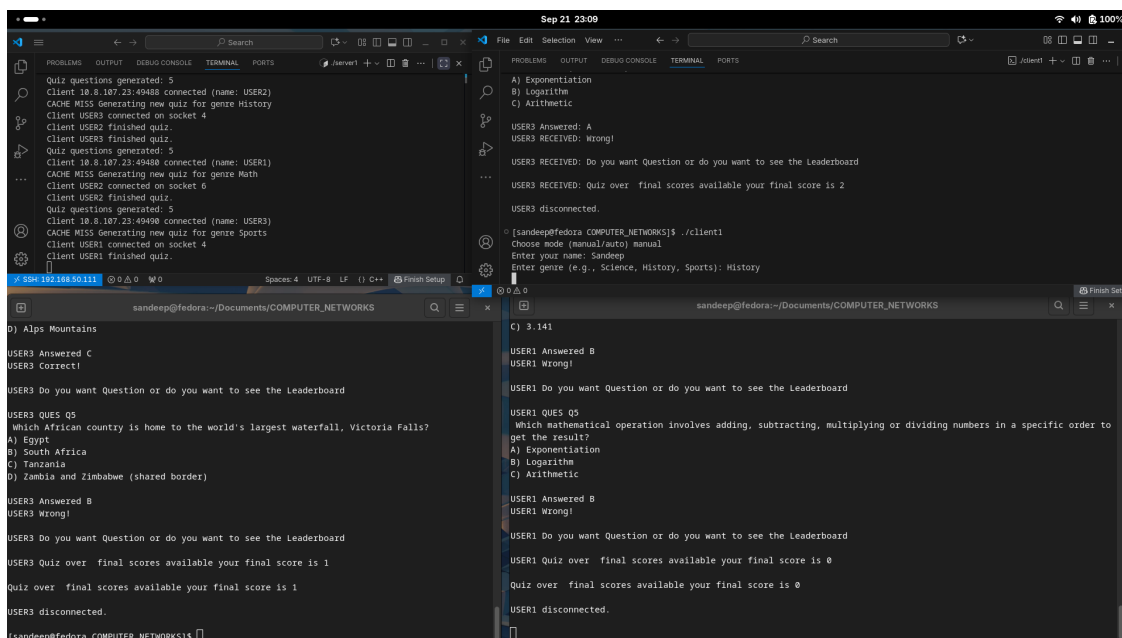
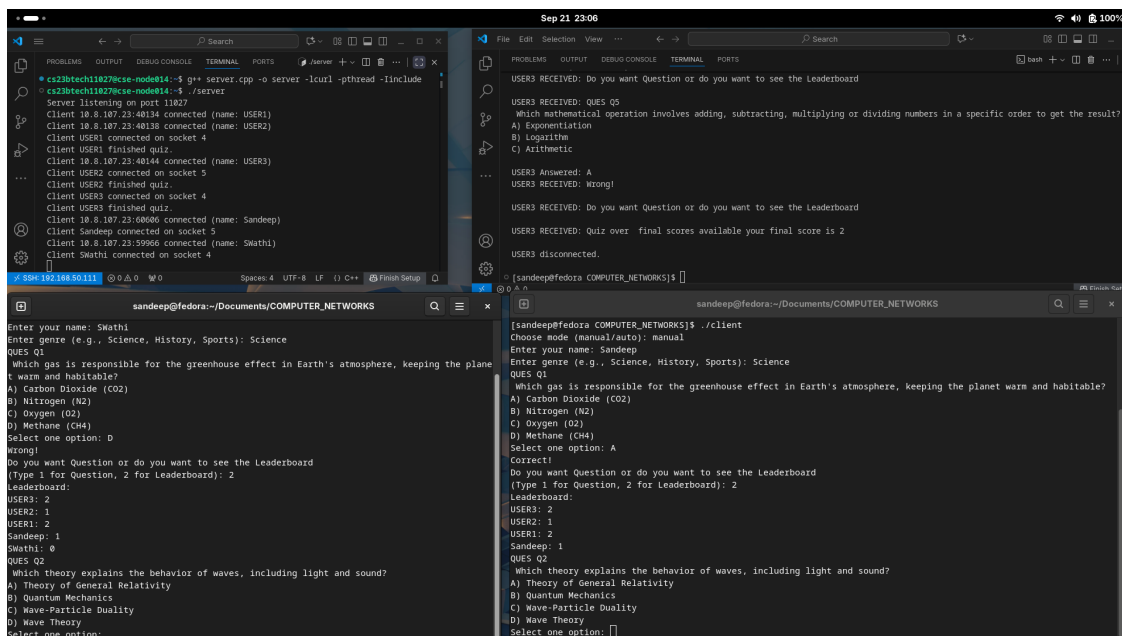


Figure 2: Server CPU utilization in the cached (Part B) scenario, showing more efficient load handling.

Server Concurrency Limit

The server reliably supports up to **9 concurrent clients**. Beyond this, latency increases due to thread management overhead. ‘pcap’ traces show delayed question deliveries, and ‘htop’ confirms near-max CPU utilization. No connection drops were observed within the 30-second test window.



Plagiarism Statement

We certify that this assignment/report is our own work based on our personal study and/or research and that we have acknowledged all material and sources used in its preparation, whether they be books, articles, packages, datasets, reports, lecture notes, and any other

kind of document, electronic or personal communication. We also certify that this assignment/report has not previously been submitted for assessment/project in any other course lab, except where specific permission has been granted from all course instructors involved, or at any other time in this course, and that we have not copied in part or whole or otherwise plagiarized the work of other students and/or persons. We pledge to uphold the principles of honesty and responsibility at CSE@IITH. In addition, we understand our responsibility to report honor violations by other students if we become aware of it.

Name: K. Sandeep

Date: 21-09-2025

Signature: Krapa.